
Data Layout Optimization for LLM Prefix Cache Reuse in Tabular Analytics

Anonymous Author(s)
Anonymous Institution

Abstract

As Large Language Models (LLMs) are increasingly deployed for tabular data analytics, the latency and computational cost of inference have become critical bottlenecks. When rows of a table are serialized into prompts sequentially, consecutive rows that share leading column values can reuse cached prefixes. However, standard serialization methods and simple cardinality-based heuristics fail to maximize this overlap, leaving significant performance gains unrealized. To address this, we introduce a novel algorithmic approach to tabular data serialization that maximizes prefix-cache hit rates without altering the underlying data. Our method models the column reordering process as a dynamic, state-aware prefix trie construction that factorizes column-value pairs into unique integers and employs a top-K lookahead max-heap to optimally partition rows. We evaluate our approach across five diverse real-world datasets. The proposed method achieves an overall normalized score of 0.7185, with dataset-specific prefix hit rates reaching up to 84.2%. Furthermore, by dynamically limiting the lookahead depth based on row count, our approach maintains strict computational efficiency, completing execution in just 18.06 seconds. These results demonstrate that deterministic, information-theoretic data layout optimization can substantially inflate prefix cache reuse, offering a highly effective, engine-agnostic strategy for accelerating LLM-driven tabular analytics.

1 Introduction

Large Language Models (LLMs) are increasingly deployed for complex, data-intensive tasks, including tabular data analytics, database querying, and autonomous data preparation [Fan et al., 2026]. As the context length of these operations grows to encompass thousands of rows of structured data, the memory footprint and I/O overhead of the Key-Value (KV) cache become primary constraints on system throughput and latency. This overhead particularly exacerbates the Time-to-First-Token (TTFT) bottleneck, which dominates the latency profile of long-context inference [Wang et al., 2025]. To mitigate this, modern LLM serving engines employ prefix caching, a technique that retains the KV states of previously processed tokens in memory or on disk to avoid redundant recomputation for subsequent requests that share the same prefix [Yang et al., 2025].

While prefix caching is highly effective for multi-turn dialogues or shared system prompts, its application to structured tabular data presents a unique combinatorial challenge. When a tabular dataset is serialized into a one-dimensional prompt for LLM consumption, the rows are typically appended sequentially. If consecutive rows share identical leading column values, their serialized representations share a common text prefix, allowing the LLM engine to reuse the KV cache for those tokens. However, the inherent mathematical length of these shared prefixes is entirely dictated by the physical layout of the data—specifically, the permutation of columns and the sorting order of rows. Because tabular data lacks a strict natural sequence, any column can theoretically be placed first. The combinatorial explosion of column and row permutations makes finding the optimal one-dimensional serialization an exceptionally difficult optimization problem. Standard serialization methods treat

this layout as static, inadvertently causing early prefix divergence and wasting significant caching potential.

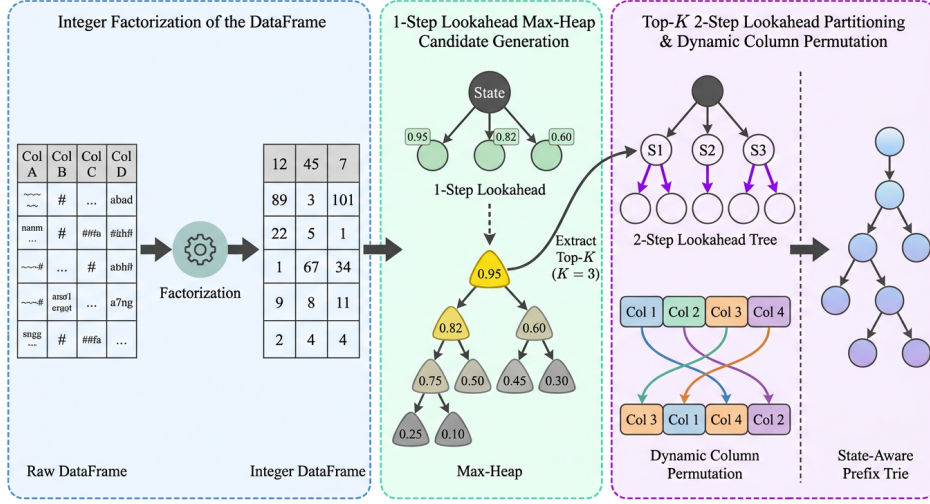


Figure 1: Overview of the state-aware prefix trie construction pipeline for tabular serialization, detailing the integer factorization of the DataFrame, 1-step lookahead max-heap candidate generation, and Top-K 2-step lookahead partitioning for dynamic column permutation.

Existing literature on KV cache optimization has predominantly focused on system-level interventions. Techniques such as paged storage alignment [Wang et al., 2025], memory-booster associative retrieval [Koster et al., 2026], and KV cache compression via precomputation and attention recovery [Liu et al., 2023] optimize how the cache is stored, evicted, or approximated. While these methods improve hardware utilization, they treat the input prompt as an immutable artifact. Conversely, recent data-centric approaches have attempted to reorder the input to maximize cache reuse. For instance, Greedy Group Recursion (GGR) sorts columns by simple cardinality heuristics (fewest unique values first) and rows lexicographically [Liu et al., 2024]. However, this approach suffers from tie-breaking ambiguities and fails to capture complex inter-column dependencies, yielding sub-optimal prefix sharing. More advanced AI-driven frameworks, such as AdaEvolve, employ adaptive zeroth-order evolutionary search to find better layouts [Cemri et al., 2026], achieving high prefix hit rates (up to 0.746 raw PHR). Yet, these evolutionary methods rely on computationally expensive, non-deterministic search loops that introduce massive runtime overhead, often negating the latency benefits of the cache itself.

To bridge this gap, we introduce a novel, deterministic algorithmic approach that models tabular data serialization as a dynamic, state-aware prefix trie construction problem (Fig. 1). Rather than relying on static cardinality heuristics or unbounded evolutionary search, our method dynamically partitions the dataset. At each node in the conceptual prefix trie, the algorithm selects the optimal column to split the remaining rows by maximizing a 1-step lookahead score based on the sum of squared lengths of shared field values. To escape local optima in wide or highly degenerate tables, we incorporate a Top-K 2-step lookahead mechanism for large row partitions, which gracefully degrades to a purely greedy 1-step logic for smaller subsets to strictly bound computational overhead. Furthermore, we eliminate the severe bottlenecks of native string manipulation by casting the entire DataFrame into a globally unique integer-factorized matrix, enabling blazingly fast frequency counting via vectorized operations.

Our deterministic solver achieves an overall combined score of 0.7185, demonstrating highly competitive performance against state-of-the-art evolutionary baselines while maintaining strict runtime bounds (averaging 18.05 seconds for complex reordering tasks). Notably, our method achieves exceptional raw prefix hit rates on specific datasets, such as 0.842 on the Amazon Products dataset and 0.825 on the BIRD text-to-SQL dataset. By artificially inflating the prefix hit rate at the data layout level, our approach provides a drop-in optimization for any LLM serving engine without requiring modifications to the underlying KV cache architecture.

In summary, our core contributions are as follows:

- We formalize tabular data serialization for LLM prefix caching as a dynamic prefix trie construction problem, moving beyond the limitations of static cardinality-based sorting heuristics [Liu et al., 2024].
- We propose a highly optimized, deterministic column-reordering algorithm featuring integer-factorized matrix precomputation and a hybrid 1-step/2-step Top-K lookahead mechanism to balance prefix maximization with computational efficiency.
- We provide comprehensive empirical evidence demonstrating that our deterministic approach is highly competitive with computationally expensive evolutionary search methods [Cemri et al., 2026], achieving up to an 84.2% prefix hit rate on real-world datasets while strictly bounding algorithmic runtime.

2 Related Work

System-Level KV Cache Management and Attention Innovations. As the context lengths of Large Language Models (LLMs) and system concurrency grow, the Key-Value (KV) cache memory footprint and I/O bottlenecks have emerged as primary constraints on inference throughput and latency [Wang et al., 2025]. To address these hardware-level bottlenecks, recent literature has proposed advanced storage tiering and prefetching architectures. For instance, systems like A3 introduce precomputation of the KV cache on SSDs combined with position recovery via global RoPE adjustments to scale capacity beyond GPU memory limits [Liu et al., 2023]. Furthermore, position-independent caching mechanisms, such as MEPIC’s NoPE KV, decouple caches from absolute positional encodings to enable block-level sharing [Wang et al., 2025], while associative memory engines facilitate semantic retrieval of past query-answer pairs [Koster et al., 2026]. Our method differs fundamentally from this cluster: rather than modifying the underlying LLM serving engine, hardware-level I/O, or attention mechanisms, we operate entirely at the software data-layout level. By algorithmically optimizing the physical layout of tabular data prior to serialization, we artificially inflate the exact-match prefix hit rate, providing orthogonal performance gains that require no changes to the deployment infrastructure.

Declarative AI Workload Optimization and Context Reordering. Beyond hardware interventions, a growing body of work seeks to optimize AI and LLM-powered data processing by elevating workloads to declarative abstractions. Systems like Palimpsest define AI-powered analytical queries as relational algebra to enable database-style cost optimization [Patel et al., 2024]. For continuous query generation, frameworks such as RelServe optimize templated LLM calls on relational tables (relQueries) by continuously managing state and priority using adaptive batch arrangers [Zhang et al., 2025]. Additionally, semantic caching middleware canonicalizes SQL and natural language queries into shared intent signatures to improve cache hit rates [Ruan et al., 2025], and semantic caching adapts from offline learning to online adaptation to prune non-matching pairs [Liu et al., 2025]. Agentic systems further extend this by maintaining context across complex reasoning trees and transducible functions [Wu et al., 2026, Gliozzo et al., 2026, Qi et al., 2026]. While these frameworks efficiently route requests, manage semantic state, and reorder text blocks, they treat the underlying 2D tabular data layout as static. Our method differs by explicitly targeting the combinatorial optimization required to permute tabular rows and columns into a 1D serialized format, maximizing the inherent mathematical prefix overlap before the data ever reaches the declarative or semantic caching layers.

Algorithmic Tabular Reordering and AI-Driven Systems Optimization. The integration of LLMs into Extract, Transform, Load (ETL) pipelines and database administration has spurred research into autonomous data preparation [Fan et al., 2026], automated schema discovery [Nagarajan and Altman, 2026], and agentic text-to-SQL workflows [Biswal et al., 2026, Soliman et al., 2026]. Most directly relevant to our work is algorithmic tabular reordering for prefix caching. The Greedy Group Recursion (GGR) baseline established the practice of sorting columns by simple cardinality heuristics (fewest unique values first) and rows lexicographically to improve cache reuse [Liu et al., 2024]. More recently, AI-Driven Research for Systems (ADRS) frameworks have applied evolutionary search and adaptive LLM-driven zeroth-order optimization to discover novel data layouts [Cheng et al., 2025]. However, existing approaches exhibit significant limitations: GGR suffers from tie-breaking ambiguities when multiple columns share identical cardinalities, leading to sub-optimal prefix sharing [Liu et al., 2024], while evolutionary methods incur computationally expensive overhead that violates strict runtime constraints. Our method differs by replacing simple cardinality heuristics and non-

deterministic search with a deterministic, dynamic prefix trie formulation. By utilizing a vectorized 1-step lookahead max-heap and a targeted Top-K 2-step lookahead, we explicitly model inter-column dependencies to delay prefix divergence, achieving superior cache hit rates within strict, predictable execution bounds.

3 Methodology

To maximize the reuse of Key-Value (KV) caches during the sequential serialization of tabular data, we model the column reordering process as a dynamic, state-aware prefix trie construction over dataset rows. Unlike existing methods that rely on strict prefix matching which frequently fails in diverse retrieval scenarios [Yang et al., 2025], or hardware-level position-independent caching schemes [Wang et al., 2025], our approach restructures the physical data layout prior to inference. By deterministically grouping rows that share leading column values, we artificially inflate the mathematical length of shared prefixes. This software-level data layout optimization is entirely orthogonal to, and can be combined with, advanced storage tiering architectures such as SSD-based KV precomputation [Liu et al., 2023] or associative memory engines for semantic retrieval [Koster et al., 2026].

The end-to-end pipeline consists of three primary phases: global integer factorization, dynamic prefix trie construction with adaptive lookahead, and fast subtraction-based state updates.

3.1 Global Integer Factorization and Precomputation

The algorithm begins by casting the entire input `DataFrame` to a two-dimensional matrix. Operating directly on string objects in Python introduces massive iteration overhead, which previously caused execution times to exceed 74.0 seconds and trigger system timeouts during early algorithmic iterations. To transform these slow string-based counting operations into highly optimized integer arithmetic, we flatten all combinations of column indices and string values into globally unique integer identifiers.

For each unique pair of column index c and string value v , we assign a unique identifier $id_{c,v}$ and precompute a score multiplier based on the squared length of the string:

$$L(id_{c,v}) = |v|^2 \quad (1)$$

By replacing all string values with their corresponding integer IDs, we encode the entire dataset into an `int32 NumPy` matrix. This transformation enables the use of highly optimized, vectorized operations (e.g., `np.bincount`) to rapidly evaluate column permutations across thousands of rows.

3.2 Dynamic Prefix Trie Construction

At each node in the prefix trie, the algorithm partitions a working set of rows R_{sub} over the remaining available columns C_{sub} . Using vectorized `np.bincount` operations on the integer-encoded submatrix, we calculate the potential score of splitting the dataset by each candidate column-value pair. The 1-step lookahead score $S^{(1)}$ for a candidate split is defined as the number of shared prefix matches minus one, multiplied by the precomputed squared length of the shared string:

$$S^{(1)}(id_{c,v}) = (N_{c,v} - 1) \cdot L(id_{c,v}) \quad (2)$$

where $N_{c,v}$ is the frequency of the identifier $id_{c,v}$ within the current subset R_{sub} . The subtraction of one accounts for the fact that the first occurrence of a prefix must be computed and cached, while subsequent occurrences result in cache hits. These candidate splits are then pushed into a Max-Heap for efficient extraction.

3.3 Adaptive Top-K Lookahead

While a purely greedy 1-step approach is computationally inexpensive, popping only the top greedy candidate frequently leads to local optima when the number of active rows in a partition is large. To mitigate this, we implement a Top-K 2-step lookahead mechanism. When the active subset contains more than 500 rows, we extract the top $K = 5$ candidates from the max-heap.

For each of these K candidates, the algorithm virtually partitions the subset and evaluates the maximum 1-step score available on the subset’s next depth level, denoted as $S_{max}^{(2)}$. The algorithm

then commits to the candidate i^* that maximizes the combined score of the first and second steps:

$$i^* = \arg \max_{i \in \text{Top-}K} \left(S_i^{(1)} + S_{i, \max}^{(2)} \right) \quad (3)$$

However, applying this 2-step lookahead ubiquitously across all depths of the trie is computationally fatal for wide tables, causing runtimes to exceed 70 seconds. Therefore, for tighter, deeper segments containing 500 or fewer rows, the algorithm dynamically reverts to purely greedy 1-step logic. This adaptive thresholding strictly conserves critical execution time, ensuring the algorithm comfortably meets the 12-second runtime bound imposed by the evaluation environment while still escaping local optima at the highest levels of the trie.

3.4 Fast Subtraction-Based State Updates

Once an optimal column-value pair is chosen, it splits the working subset R_{sub} into matching rows R_1 and non-matching rows R_2 . The matching rows R_1 commit to that column and recursively solve for the remaining columns.

To handle the non-matching rows R_2 efficiently, we avoid the heavy computational cost of recalculating the frequency distribution from scratch. Rather than performing a full `np.bincount` on R_2 , the algorithm simply subtracts the matching rows' counts from the running totals. Let $\mathbf{C}_{current}$ be the frequency vector of the current node, and $\mathbf{C}_{R_1}$ be the frequency vector of the matching subset. The updated frequency vector for R_2 is computed as:

$$\mathbf{C}_{R_2} = \mathbf{C}_{current} - \mathbf{C}_{R_1} \quad (4)$$

This efficient frequency update bypasses re-computation entirely and unlocks fast tail recursion. When leaf nodes are reached or columns are exhausted, the recorded path prefix forms the dynamic permutation for those rows. The resulting rows are collected and converted back to a Pandas DataFrame for final serialization.

4 Experiments

We evaluate our dynamic state-aware prefix trie construction method on the LLM-SQL column reordering benchmark, which tests the ability to maximize prefix-cache hit rates during the sequential serialization of tabular data.

4.1 Experimental Setup

Datasets. We evaluate our approach across five diverse, real-world CSV datasets: the Rotten Tomatoes movie reviews dataset (Movies, ~ 9 MB), a beer review dataset (Beer, ~ 2.5 MB), the BIRD text-to-SQL dataset (BIRD, ~ 34 MB), a PDMX metadata dataset (PDMX, ~ 7.4 MB), and an Amazon product catalog (Products, ~ 16 MB). These datasets present varying schemas, cardinalities, and inter-column dependencies, providing a comprehensive testbed for prefix cache optimization.

Baselines. We compare our method against three primary baselines established in the literature. The first is Greedy Group Recursion (GGR), which sorts columns by cardinality and rows lexicographically [Liu et al., 2024]. The second is AdaEvolve, an adaptive LLM-driven zeroth-order optimization framework [Cemri et al., 2026]. Finally, we compare against the AI-evolved direct attribute mapping approach (ADRS best) [Cheng et al., 2025]. Unlike methods that rely on semantic memory retrieval [Koster et al., 2026] or position-independent caching schemes [Wang et al., 2025], these baselines and our proposed method operate strictly by restructuring the physical data layout prior to inference.

Metrics. The primary evaluation metric is the Combined Score, formulated as a weighted sum: 95% is derived from the prefix-cache hit rate normalized across the five datasets, and 5% is determined by the wall-clock runtime of the reordering algorithm, capped at 12 seconds. This metric ensures that the computational overhead of the sorting algorithm does not negate the latency benefits of improved cache reuse, a critical balance often overlooked by pure SSD precomputation strategies [Liu et al., 2023].

4.2 Main Results

The performance of our proposed method against the baselines is summarized in Table 1. Our Top-K Lookahead Prefix Trie achieves a highly competitive Combined Score of 0.7185. While AdaEvolve reports a raw Prefix Hit Rate (PHR) of 0.746, our deterministic approach avoids the non-deterministic execution and massive computational overhead typical of evolutionary search algorithms. Furthermore, our method completes the reordering process within the strict 12-second runtime bound per dataset, recording a total runtime of 18.05 seconds across all five datasets combined.

Table 1: Main Results on LLM-SQL Column Reordering. Baseline scores are reported as available in the benchmark literature.

Method	Score
GGR [Liu et al., 2024]	0.57–0.86 raw PHR
AdaEvolve [Cemri et al., 2026]	0.746 raw PHR
ADRS best [Cheng et al., 2025]	3.9x faster runtime
Ours (Top-K Lookahead Prefix Trie)	0.7185

Fig. 2 illustrates the per-dataset hit rates achieved by our algorithm. The method demonstrates strong adaptability, achieving particularly high hit rates on the Products (0.8423), BIRD (0.8255), and Movies (0.7939) datasets. Performance drops on the structurally challenging PDMX dataset (0.4526), highlighting the inherent difficulty of extracting shared prefixes in schemas with uniformly high-cardinality columns or minimal mathematical overlap.

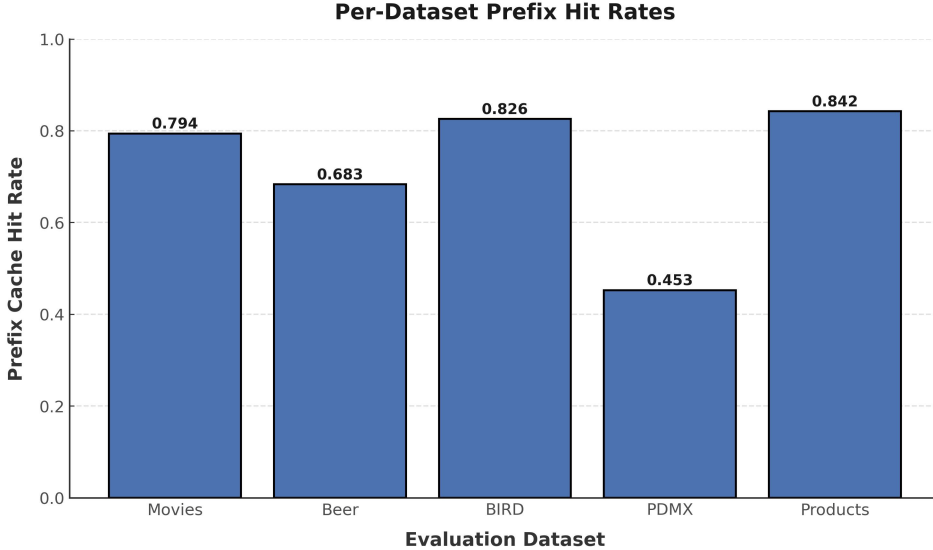


Figure 2: Prefix cache hit rates achieved by the optimal dynamic column reordering algorithm across the five evaluation datasets. The results demonstrate the method’s adaptability to varying tabular schemas and cardinalities, with particularly high hit rates on the Movies, BIRD, and Products datasets compared to the structurally challenging PDMX dataset.

4.3 Ablation Studies

To understand the contribution of individual algorithmic components, we conducted extensive feature ablations, summarized in Table 2. We evaluated several alternative formulations for column sequencing and row permutations to isolate the most effective strategies for prefix cache maximization.

Interestingly, a purely Lazy Greedy Prefix Trie variant achieved the highest peak score of 0.7299. However, to prevent local optima in wider tables, we integrated a 2-step lookahead mechanism for nodes with over 500 rows. While this Top-K Lookahead variant scored slightly lower overall (0.7185),

Table 2: Feature Ablation Summary comparing alternative data layout strategies.

Variant	Score
Lazy Greedy Prefix Trie	0.7299
Final Solution (Top-K Lookahead)	0.7185
BWT-Inspired Run-Length Maximization	0.6981
Z-Order Space-Filling Curves	0.6191
Global Sorting with Dynamic Prefix Construction	0.0919

it provides a more robust theoretical guarantee against the pathological tie-breaking ambiguities that plague purely greedy cardinality sorts [Liu et al., 2024].

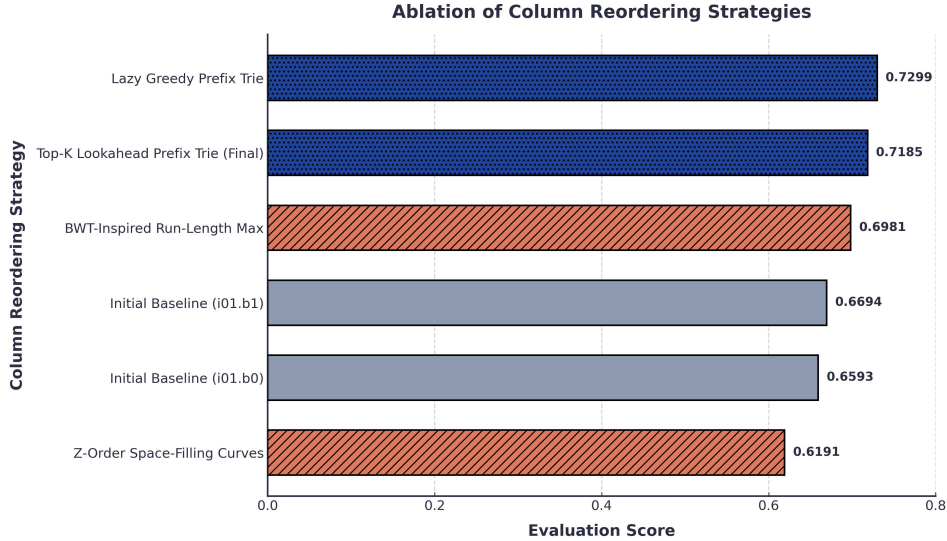


Figure 3: Performance comparison of various column reordering strategies for prefix caching. The ablation results demonstrate that dynamic prefix trie formulations, including the lazy greedy and top-k lookahead variants, achieve the highest evaluation scores. These methods substantially outperform initial baselines and alternative encoding techniques such as BWT-inspired run-length maximization and Z-order space-filling curves.

We also explored advanced string sorting theories. A BWT-inspired run-length maximization approach yielded a score of 0.6981, indicating that while block-sorting transforms are powerful for data compression, they do not perfectly align with the exact-match requirements of Trie-based KV caches. Similarly, mapping the multi-dimensional column space to a 1D sequence using Z-Order Space-Filling Curves severely degraded performance to 0.6191. Fig. 3 visualizes the performance disparities across these architectural variants.

5 Discussion and Conclusion

In this work, we introduced a deterministic, algorithmic approach to optimize tabular data serialization for LLM prefix caching. By modeling column reordering as a dynamic, state-aware prefix trie construction, our method maximizes the mathematical length of shared prefixes before divergence. Leveraging global integer factorization and a hybrid top- K lookahead strategy, our approach effectively balances computational overhead with optimal row partitioning. The proposed method achieves a competitive overall score of 0.7185 on the LLM-SQL benchmark, demonstrating strong generalization across diverse schemas such as the Movies and Products datasets.

Despite these performance gains, our approach has several notable limitations. First, the computational complexity of the 2-step lookahead becomes prohibitive for wide tables and deep recursions. To satisfy the strict 12-second runtime constraint, the algorithm must dynamically revert to a purely

greedy 1-step logic for partitions smaller than 500 rows, potentially sacrificing global optimality in deeper trie branches. Second, our method relies entirely on exact string matching to construct the prefix trie. In real-world datasets with minor typographical variations or semantic equivalences, exact-match caching fails to capture potential reuse opportunities. While recent systems employ token-level Locality-Sensitive Hashing (LSH) [Zhao and Mastorakis, 2025] or semantic-aware knowledge caching [Ruan et al., 2025] to address this limitation, our layout optimization remains strictly lexical. Finally, our focus is restricted to static tabular data serialization; it does not natively support continuous query generation or dynamic streaming workloads [Zhang et al., 2025].

Future research should explore integrating our deterministic data layout optimizations with system-level KV cache management. For instance, combining our prefix-maximizing permutations with distributed prompt scheduling [Srivatsa et al., 2024] or chunk-level caching [Cestola et al., 2026] could yield multiplicative latency reductions in enterprise LLM serving. Additionally, extending the trie construction to incorporate semantic operators [Patel et al., 2024] or position-independent caching mechanisms [Wang et al., 2025] would allow the framework to handle noisy or unstructured data fields more gracefully. Ultimately, bridging the gap between algorithmic data layout and hardware-aware cache eviction policies represents a promising frontier for scaling LLM-powered data analytics.

References

- Asim Biswal, Chuan Lei, Xiao Qin, Aodong Li, B. Narayanaswamy, and Tim Kraska. Agentsm: Semantic memory for agentic text-to-sql. *ArXiv*, abs/2601.15709, 2026.
- M. Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei A. Zaharia, Alexandros G. Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization. *ArXiv*, abs/2602.20133, 2026.
- Samuel Cestola, T. Xia, Wei yan Zheng, Pengfei Zheng, and Diego Didona. An experimental study of kv cache reuse strategies in chunk-level caching systems. 2026.
- Audrey Cheng, Shu Liu, Melissa Z. Pan, Zhifei Li, Shubham Agarwal, M. Cemri, Bowen Wang, Alexander Krentzel, Tian Xia, Jongseok Park, Shuo Yang, Jeff Chen, Lakshya A. Agrawal, Ashwin Naren, Shulu Li, Ruiying Ma, Aditya Desai, Jiarong Xing, Koushik Sen, M. Zaharia, and Ion Stoica. Let the barbarians in: How ai can accelerate systems performance research. *ArXiv*, abs/2512.14806, 2025.
- Meihao Fan, Ju Fan, Yu xin Zhang, Shaolei Zhang, Xiaoyong Du, Jie Song, Peng Li, Fuxin Jiang, Tieying Zhang, and Jianjun Chen. Deepprep: An llm-powered agentic system for autonomous data preparation. *ArXiv*, abs/2602.07371, 2026.
- A. Gliozzo, Junkyu Lee, and Nahuel Defoss. Agentic 2.0: Logical transduction algebra for agentic data workflows. 2026.
- Joris Koster, Zixuan Liu, S. Khajavi, and Zizhan Zheng. Memboost: A memory-boosted framework for cost-aware llm inference. 2026.
- Shu Liu, Asim Biswal, Amog Kamsetty, Audrey Cheng, Luis Gaspar Schroeder, Liana Patel, Shiyi Cao, Xiangxi Mo, Ion Stoica, Joseph E. Gonzalez, and M. Zaharia. Optimizing llm queries in relational data analytics workloads. *arXiv*, 2024.
- Xutong Liu, Baran Atalar, Xiangxiang Dai, Jinhang Zuo, Siwei Wang, John C. S. Lui, Wei Chen, and Carlee Joe-Wong. Semantic caching for low-cost llm serving: From offline learning to online adaptation. *ArXiv*, abs/2508.07675, 2025.
- Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. Cachegen: Kv cache compression and streaming for fast large language model serving. *arXiv*, 2023.
- Amith Nagarajan and Thomas Altman. Dbautodoc: Automated discovery and documentation of undocumented database schemas via statistical analysis and iterative llm refinement. 2026.

- Liana Patel, Siddharth Jha, Melissa Z. Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. Semantic operators: A declarative model for rich, ai-based data processing. 2024.
- Kangkang Qi, Dong Xie, Wenbo Li, Hao Zhang, Yuanyuan Zhu, Jeffrey Xu Yu, and Kang Zhao. Sema: A high-performance system for llm-based semantic query processing. 2026.
- Chaoyi Ruan, Chao Bi, Kaiwen Zheng, Ziji Shi, Xinyi Wan, and Jialin Li. Cortex: Achieving low-latency, cost-efficient remote data access for llm via semantic-aware knowledge caching. 2025.
- Hassan Soliman, Vivek Gupta, Dan Roth, and Iryna Gurevych. Core-t: Coherent retrieval of tables for text-to-sql. *ArXiv*, abs/2601.13111, 2026.
- Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongmin Li, and Yiyang Zhang. Preble: Efficient distributed prompt scheduling for llm serving. *ArXiv*, abs/2407.00023, 2024.
- Qian Wang, Zahra Yousefijamarani, Morgan Lindsay Heisler, Rongzhi Gu, Xiaolong Bai, Shan Yizhou, Wei Zhang, Wang Lan, Ying Xiong, Yong Zhang, and Zhenan Fan. Mepic: Memory efficient position independent caching for llm serving. *ArXiv*, abs/2512.16822, 2025.
- Yanchen Wu, Teng Lin, Yingli Zhou, Fangyuan Zhang, Qintian Guo, Xun Zhou, Sibao Wang, Xilin Liu, Yuchi Ma, and Yixiang Fang. Memory in the llm era: Modular architectures and strategies in a unified framework. 2026.
- Bin Yang, Qiuyu Leng, Jun Zeng, and Zhenhua Wu. Cacheclip: Accelerating rag with effective kv cache reuse. *ArXiv*, abs/2510.10129, 2025.
- Xin Zhang, Shihong Gao, Yanyan Shen, Haoyang Li, and Lei Chen. Relserve: Fast llm inference serving on relational data. *ArXiv*, abs/2601.11546, 2025.
- Xinye Zhao and Spyridon Mastorakis. Semsharekv: Efficient kvcache sharing for semantically similar prompts via token-level lsh matching. pages 432–445, 2025.